Voice Assistant: Project Documentation

A fully on-device voice assistant: speech-to-text, a local LLM, and text-to-speech, with document Q&A (RAG) and a small set of system tools. No cloud services are used.

1. Overview

Pipeline

```
Mic -> Whisper (faster-whisper) -> LM Studio (local LLM) -> Piper TTS -> Speaker
```

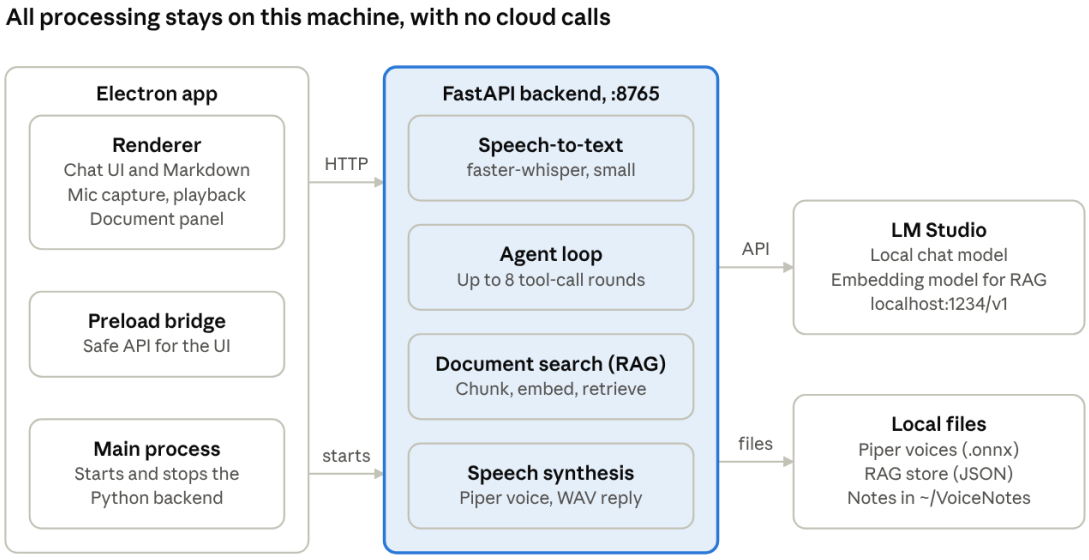
The project ships three interfaces over the same idea:

Interface	Entry point	Capabilities
Terminal	`listen.py`	Push-to-talk (Enter to start/stop), conversation history, spoken replies
Gradio web UI	`app.py`	Browser mic recording, chat view, spoken replies (legacy)
Electron desktop app	`electron/` + `server.py`	Full feature set: voice and text input, agent tools, RAG over documents, multiple chat sessions, voice switching, playback speed

The Electron app plus the FastAPI backend (`server.py`) is the primary, most complete interface. The terminal and Gradio versions are simpler, standalone alternatives without tools or RAG.

2. Architecture

2.1 High-level diagram



architecture · app, backend, local services

The Electron app talks to the backend over local HTTP; the backend calls LM Studio for the model and reads and writes local files.

2.2 Components

Electron main process (`electron/main.js`)

- Spawns the backend: `venv` Python running `uvicorn server:app` on `127.0.0.1:8765` (single worker).
- Forwards Python stdout/stderr to the terminal and to the renderer DevTools console (`py-log`).
- Provides the app menu (Edit, View, Server) including a *Restart Python Server* action.
- Terminates the Python process when the app quits.

Preload bridge (`electron/preload.js`)

With `contextIsolation` on and `nodeIntegration` off, it exposes only: `getServerUrl()`, `onPyLog(cb)`, `copyToClipboard(text)`, `readClipboard()`.

Renderer (`electron/renderer/`)

- Records microphone audio as Float32 PCM and posts it to `/transcribe`, then sends the transcript to `/chat`.
- Supports typed input as well as voice.
- Renders assistant replies as Markdown (`marked`), with tool-activity and document-source hints under each reply.
- Keeps multiple in-memory chat sessions; only the last 10 messages are sent as context.

- Settings: playback speed (1x to 3x), read-aloud toggle, stop-speaking button, voice selector.
- A Content-Security-Policy restricts network access to `http://127.0.0.1:8765`.

Backend (`server.py`)

- **STT:** `faster-whisper` model `small`, CPU, int8. Audio is resampled to 16 kHz; VAD filtering is enabled.
- **LLM client:** OpenAI SDK pointed at LM Studio (`http://localhost:1234/v1`).
- **TTS:** Piper voice loaded from the platform data directory; the reply is cleaned of Markdown and returned as a base64 WAV.
- **RAG:** `VectorStore` keeps one JSON file per document (chunks plus embeddings) and searches by cosine similarity.
- **Agent loop:** tool-calling loop with a maximum of 8 rounds before a final answer is forced.

3. Tools and Technologies

3.1 Technology stack

Layer	Technology	Purpose
Speech-to-text	faster-whisper (CTranslate2), model `small`	Local transcription
LLM runtime	LM Studio (OpenAI-compatible server)	Local chat completion, tool calling, embeddings
Text-to-speech	Piper (`piper-tts`, ONNX voices)	Local speech synthesis
Backend	Python, FastAPI, Uvicorn	HTTP API
Desktop shell	Electron 35	Native window, process management
Frontend	HTML, CSS, vanilla JS, marked	Chat UI and Markdown rendering
Web UI (legacy)	Gradio	Browser-based alternative
Audio	sounddevice, SciPy, NumPy, Web Audio API	Capture, resampling, playback
Documents / RAG	PyMuPDF, NumPy	PDF text extraction, cosine-similarity search
System info	psutil	CPU, RAM, disk metrics

3.2 Agent tools (voice-invokable)

Tool	What it does	Safeguards

Tool	What it does	Safeguards
`get_datetime`	Current date, time, weekday	None needed
`get_system_info`	CPU %, RAM, disk usage	Read-only
`list_directory`	Lists up to 60 entries of a folder	Home directory only
`search_files`	Glob search (max 25 results)	Home directory only
`read_file`	Reads a text or Markdown file	Home only; files over 50 KB rejected; output truncated to 4,000 chars
`pkg_search`	Searches packages (apt, brew, or winget)	Read-only
`list_installed`	Lists installed packages, optional filter	Read-only
`create_note`	Saves a Markdown note in `~/VoiceNotes/`	Filename sanitised
`move_file`	Moves or renames a file	Home only; never overwrites an existing file

The system prompt tells the model to *advise* install commands rather than run privileged commands itself.

4. Use Cases

1. **Hands-free question answering.** Ask general questions by voice and hear the answer spoken, entirely offline and private.
2. **Chat with your documents.** Attach PDFs, text, or Markdown files (reports, lecture notes, manuals) and ask questions about them; replies show which files were used.
3. **Voice-driven file management.** "Find my PDF files", "list my Downloads folder", "read this file", "move this file to Documents".
4. **Note taking by voice.** "Create a note called meeting-notes with these action items", saved as Markdown in `~/VoiceNotes`.
5. **System monitoring.** "How much RAM and disk space am I using?"
6. **Software lookup.** Search for a package and get the exact install command for the platform's package manager.
7. **Privacy-sensitive environments.** All processing stays on the machine, which suits confidential documents or offline use.
8. **Accessibility and eyes-free use.** Spoken output, adjustable playback speed, and typed input as a fallback.

5. Setup and Running

Install LM Studio, load a chat model, and start its local server; for document Q&A, load an embedding model as well.

Create a Python virtual environment, install the requirements, and add Piper voice files. Then start the Electron app with npm, which launches the backend itself; the terminal and Gradio versions run straight from their Python scripts.

6. Configuration

Setting	Value	Location
Whisper model	`small` (server, terminal), `base` (Gradio)	`server.py`, `listen.py`, `app.py`
LM Studio URL	`http://localhost:1234/v1`	all Python entry points
Backend port	`8765`	`server.py`, `electron/main.js`, `electron/preload.js`
Gradio port	`7860`	`app.py`, `launch.sh`
Default voice	`en_GB-jenny_dioco-medium.onnx`	`server.py`, `listen.py`
RAG top-k / threshold	`8` / `0.15`	`RAG_K`, `RAG_THRESHOLD` in `server.py`
Max agent iterations	`8`	`AGENT_MAX_ITER` in `server.py`
Notes folder	`~/VoiceNotes`	`NOTES_DIR` in `server.py`

Data locations

Platform	Voices (`piper/`) and RAG store (`voice_assistant_rag/`)
Linux	`~/.local/share/`
macOS	`~/Library/Application Support/`
Windows	`%LOCALAPPDATA%`